Транзакции и ACID. Обработка ошибок и блокировка данных. Курсоры.

старший преподаватель кафедры АСУ
Валова Анастасия Александровна

Транзакция

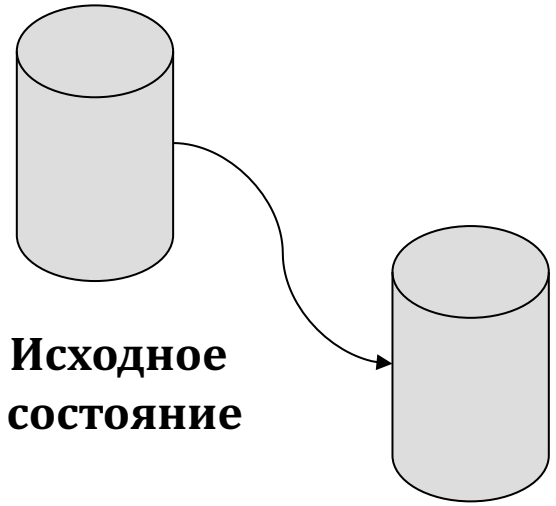
неделимая с точки зрения воздействия на БД последовательность операторов манипулирования данными (чтения, удаления, вставки, модификации) такая, что:

- либо результаты всех операторов, входящих в транзакцию, отображаются в БД;
- либо воздействие всех этих операторов полностью отсутствует.

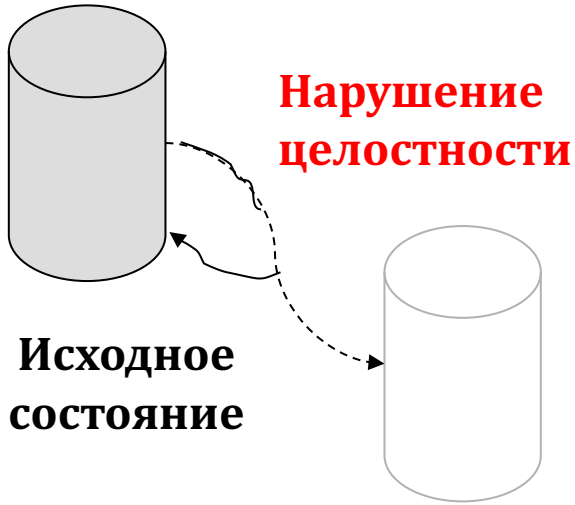
Концепция «всё или ничего»

```
UPDATE account SET balance = balance - 100 WHERE account_id = 1;
UPDATE account SET balance = balance + 100 WHERE account_id = 2;
```

Подтверждение и откат транзакции



COMMIT

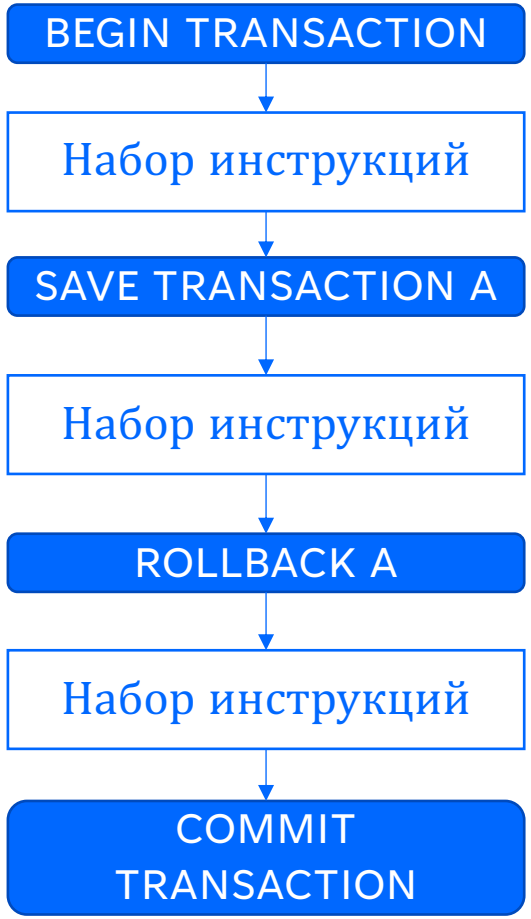


ROLLBACK

Модель управляемого выполнения транзакций

- Выбор счёта и суммы для перевода
- Проверка баланса

- Перевод со счёта отправителя
- Зачисление на счёт получателя



- **BEGIN TRANSACTION** инициирует начало транзакции;
- **COMMIT TRANSACTION** подтверждает успешное выполнение транзакции (новая транзакция не начинается автоматически);
- **SAVE TRANSACTION** создает внутри транзакции *точку сохранения* (к ней можно откатиться);
- **ROLLBACK** отменяет выполнение текущей транзакции и возвращает БД к состоянию начала транзакции или к состоянию из SAVE TRANSACTION.

https://learn.microsoft.com/ru-ru/SQL/t-sql/language-elements/transactions-transact-sql?view=azuresqldb-mi-current

Восстановление БД

(последнее согласованное состояние после сбоя)
=
Журнал транзакций
+
Архивная копия

Журнал транзакций поддерживает следующие операции:

- восстановление отдельных транзакций;
- восстановление всех незавершенных транзакций при запуске SQL Server;
- накат восстановленной базы данных, файла, файловой группы или страницы до момента сбоя;
- поддержка репликации транзакций.

Свойства транзакций ACID
(Atomicity, Consistency, Isolation, Durability)

- ***атомарность*** – операции транзакции образуют неразделимый атомарный блок с определенным началом и концом. Этот блок либо выполняется от начала до конца, либо не выполняется вообще;
- ***согласованность*** – по завершении транзакции все задействованные объекты находятся в согласованном состоянии;
- ***изолированность*** – одновременный доступ транзакций различных приложений к разделяемым объектам координируется таким образом, чтобы эти транзакции не влияли друг на друга;
- ***долговременность (устойчивость)*** – все изменения данных, осуществленные в процессе выполнения транзакции, не могут быть потеряны.

Теорема CAP (теорема Брюера)

Эмпирическое утверждение

в любой реализации распределённых вычислений возможно обеспечить не более двух из трёх следующих свойств:

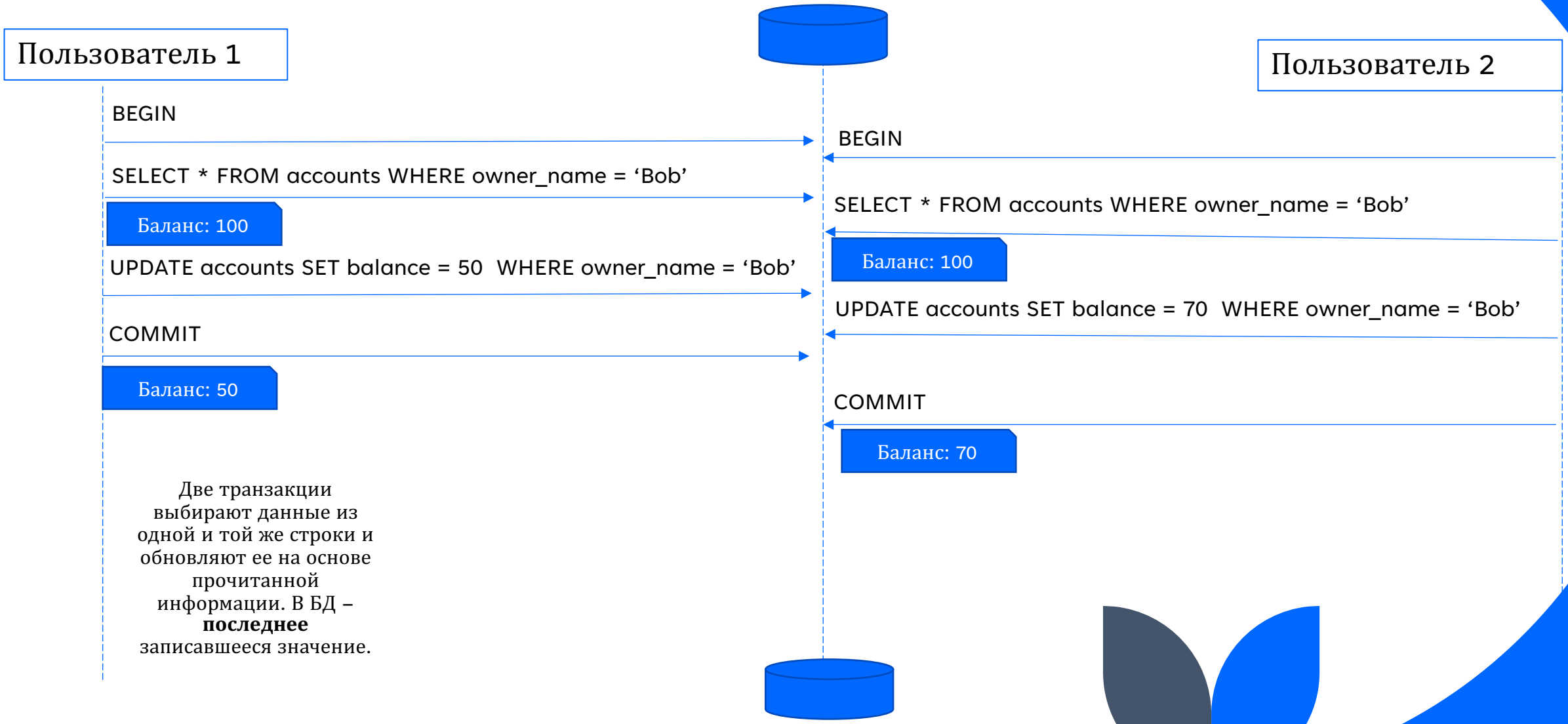
- ***согласованность (Consistency)*** — во всех вычислительных узлах в один момент времени данные не противоречат друг другу;
- ***доступность (Availability)*** — любой запрос к распределённой системе завершается корректным откликом, однако без гарантии, что ответы всех узлов системы совпадают;
- ***устойчивость к разделению (Partition tolerance)*** — расщепление распределённой системы на несколько изолированных секций не приводит к некорректности отклика от каждой из секций.

Распределённые системы распадаются на три класса — **CA**, **CP**, **AP**.

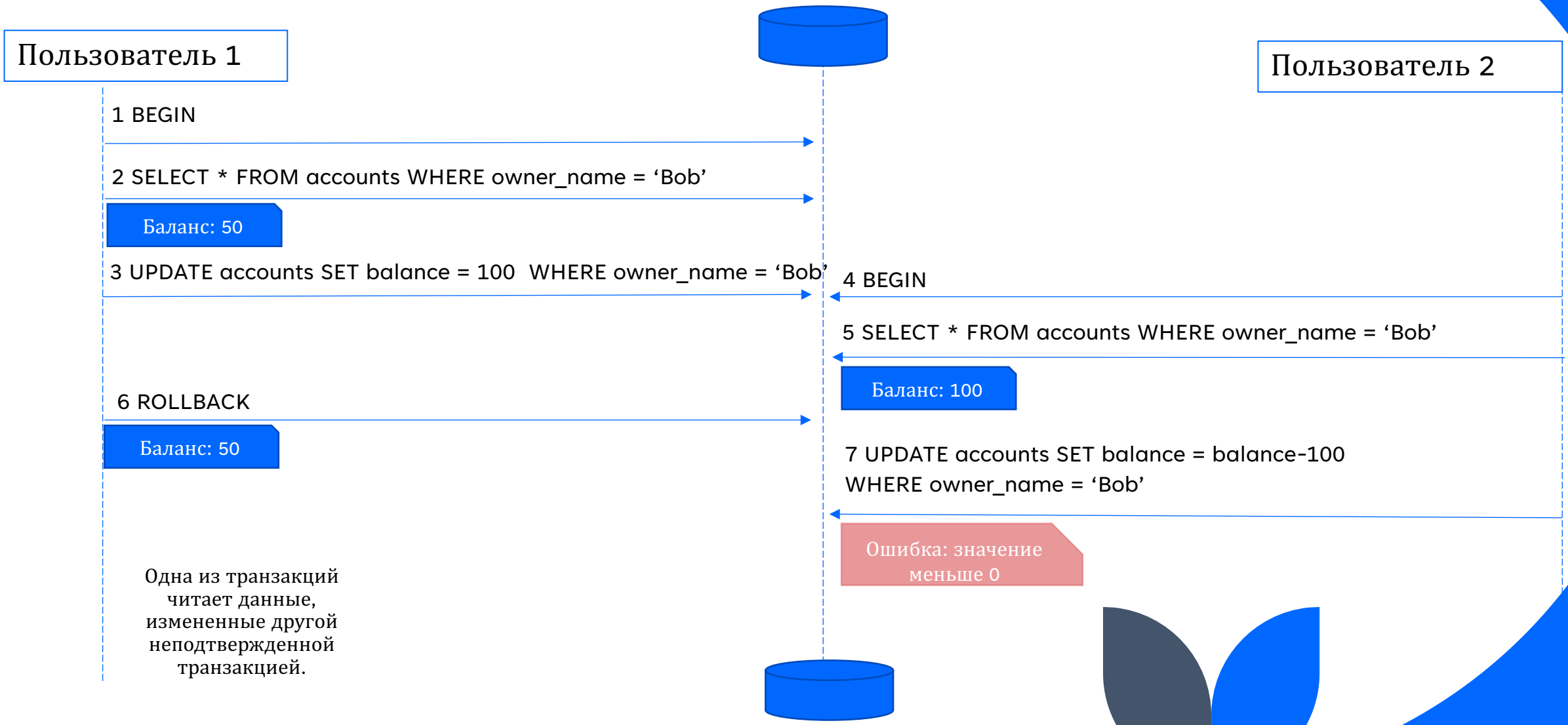
Проблемы параллелизма (concurrency)

- Пропавшие обновления (Lost Updates);
- Чтение «грязных» данных («Dirty» Reads);
- Строки-призраки (Phantom Reads).

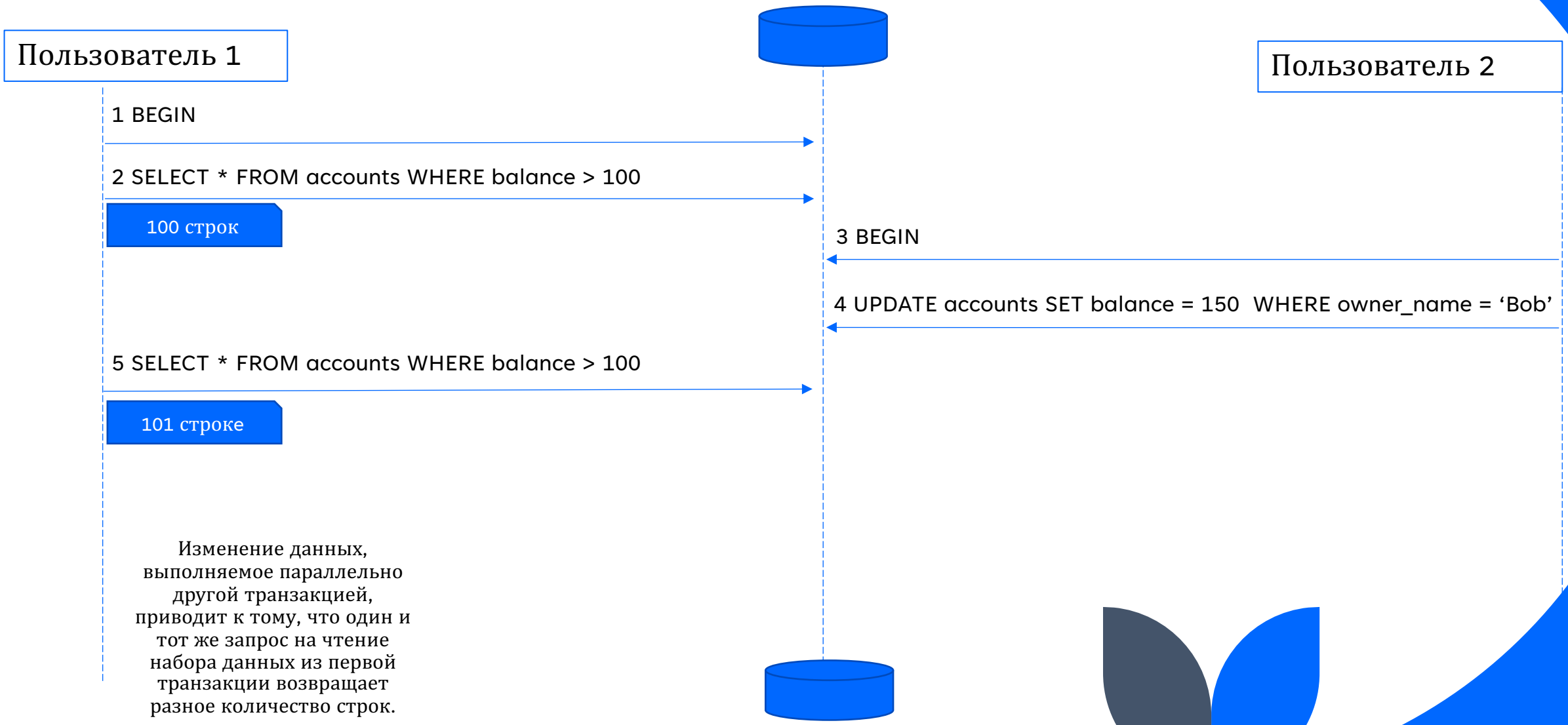
Пропавшие обновления (Lost Updates)



Чтение «грязных» данных (Dirty Reads)



Строки-призраки (Phantom Reads)



Изменение данных, выполняемое параллельно другой транзакцией, приводит к тому, что один и тот же запрос на чтение набора данных из первой транзакции возвращает разное количество строк.

Сериализация транзакций =

механизм выполнения транзакций таким образом, когда результат совместного выполнения транзакций эквивалентен результату некоторого последовательного выполнения этих же транзакций.

Режимы захватов (блокировок)

- **Совместный режим** - S (Shared), означающий разделяемый захват объекта и необходимый для выполнения операции чтения объекта.
- **Монопольный режим** - X (eXclusive), означающий монопольный захват объекта и необходимый для выполнения операций записи, удаления и модификации.

	NL	SCH-S	SCH-M	S	U	X	IS	IU	IX	SIU	SIX	UIX	BU	RS-S	RS-U	RI-N	RI-S	RI-U	RI-X	RX-S	RX-U	RX-X
NL	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N
SCH-S	N	N	C	N	N	N	N	N	N	N	N	N	N	I	I	I	I	I				
SCH-M	N	C	C	C	C	C	C	C	C	C	C	C	C	I	I	I	I	I				
S	N	N	C	N	N	C	N	N	C	N	C	C	C	N	N	N	N	N				
U	N	N	C	N	C	C	N	C	C	C	C	C	C	N	C	N	N	C				
X	N	N	C	C	C	C	C	C	C	C	C	C	C	C	C	N	C	C				
IS	N	N	C	N	N	C	N	N	N	N	N	N	C	I	I	I	I	I				
IU	N	N	C	N	C	C	N	N	N	N	N	C	C	I	I	I	I	I				
IX	N	N	C	C	C	C	N	N	N	C	C	C	C	I	I	I	I	I				
SIU	N	N	C	N	C	C	N	N	C	N	C	C	C	I	I	I	I	I				
SIX	N	N	C	C	C	C	N	N	C	C	C	C	C	I	I	I	I	I				
UIX	N	N	C	C	C	C	N	C	C	C	C	C	C	I	I	I	I	I				
BU	N	N	C	C	C	C	C	C	C	C	C	C	N	I	I	I	I	I				
RS-S	N	I	I	N	N	C	I	I	I	I	I	I	I	N	N	C	C	C				
RS-U	N	I	I	N	C	C	I	I	I	I	I	I	I	N	C	C	C	C				
RI-N	N	I	I	N	N	N	I	I	I	I	I	I	I	C	C	N	N	N				
RI-S	N	I	I	N	N	C	I	I	I	I	I	I	I	C	C	N	N	N				
RI-U	N	I	I	N	C	C	I	I	I	I	I	I	I	C	C	N	N	C				
RI-X	N	I	I	C	C	C	I	I	I	I	I	I	I	C	C	N	C	C				
RX-S	N	I	I	N	N	C	I	I	I	I	I	I	I	C	C	C	C	C				
RX-U	N	I	I	N	C	C	I	I	I	I	I	I	I	C	C	C	C	C				
RX-X	N	I	I	C	C	C	I	I	I	I	I	I	I	C	C	C	C	C				

Key

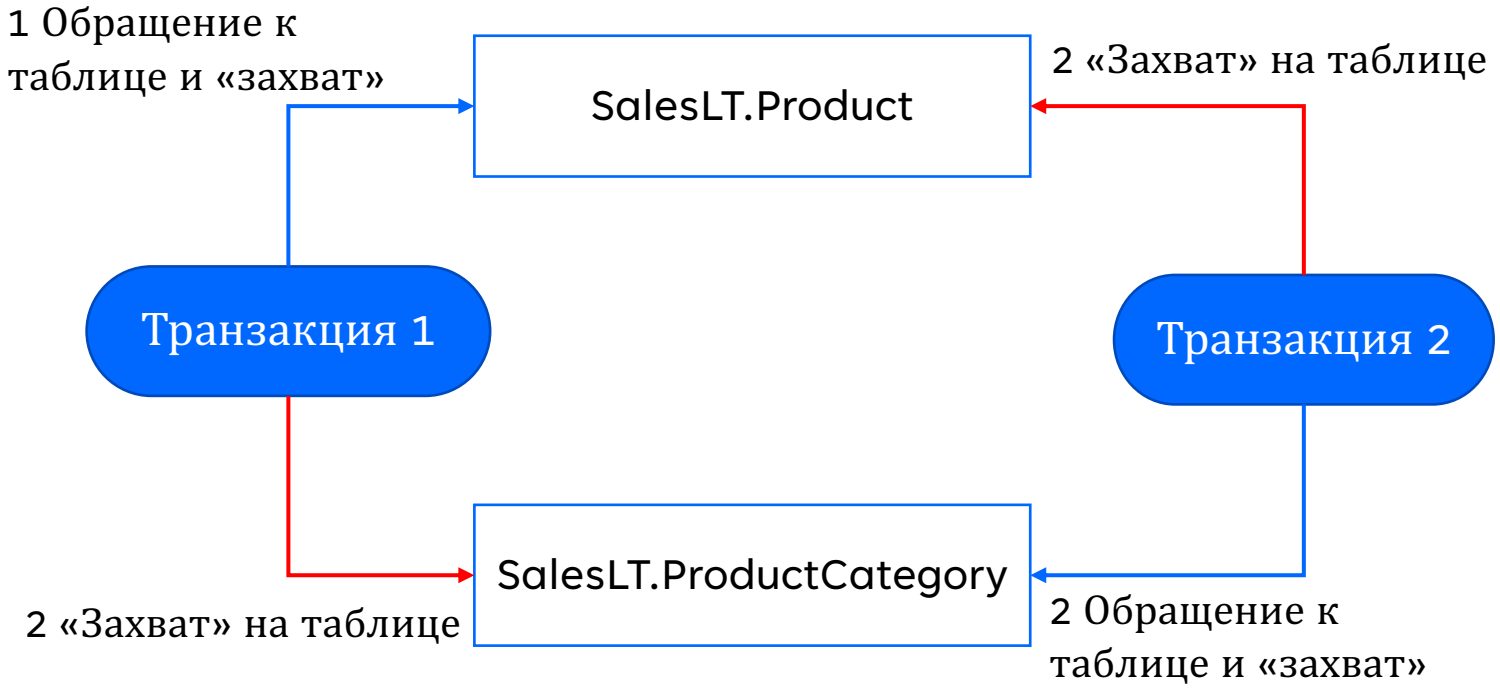
N	No Conflict	SIU	Share with Intent Update
I	Illegal	SIX	Shared with Intent Exclusive
C	Conflict	UIX	Update with Intent Exclusive
		BU	Bulk Update
NL	No Lock	RS-S	Shared Range-Shared
SCH-S	Schema Stability Locks	RS-U	Shared Range-Update
SCH-M	Schema Modification Locks	RI-N	Insert Range-Null
S	Shared	RI-S	Insert Range-Shared
U	Update	RI-U	Insert Range-Update
X	Exclusive	RI-X	Insert Range-Exclusive
IS	Intent Shared	RX-S	Exclusive Range-Shared
IU	Intent Update	RX-U	Exclusive Range-Update
IX	Intent Exclusive	RX-X	Exclusive Range-Exclusive

https://docs.microsoft.com/ru-ru/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?

Захват и освобождение объекта

Двухфазный протокол захватов объектов БД:

- первая фаза транзакции - накопление захватов;
- вторая фаза транзакции (фиксация или откат) - освобождение захватов.



Взаимные блокировки

Режимы изоляции транзакций SQL Server

- **READ UNCOMMITTED**
  - Допускает все проблемы параллелизма
- **READ COMMITTED**
  - По умолчанию. Предотвращает чтение «грязных» данных (dirty reads).
- **REPEATABLE READ**
  - Предотвращает чтение несогласованных данных. Допускает только строки-призраки (фантомное чтение).
- **SNAPSHOT**
  - Использует контроль версий строки (хранит в TempDB). Похож на SERIALIZABLE, но с меньшим количеством блокировок.
- **SERIALIZABLE**
  - Самый безопасный, не допускаются строки-призраки (фантомное чтение).

Современные тенденции

1. **Доминирование MVCC (многоверсионное управление параллельным доступом)** большинство СУБД используют контроль версий вместо блокировок
2. **SNAPSHOT** стал стандартом для высоких требований изоляции
3. **REPEATABLE READ** остается сильным в MySQL, но теряет популярность в других СУБД
4. **SERIALIZABLE** стал производительнее за счет оптимистических подходов

11 Обработка ошибок и транзакции

Ошибки и сообщения об ошибках

Elements of Database Engine Errors	
Error number	Уникальный номер, определяющий конкретную ошибку
Error message	Текст, описывающий ошибку
Severity	Числовая индикация серьезности от 1-25
State	Код внутреннего состояния ядра базы данных
Procedure	Имя хранимой процедуры или триггера, в котором произошла ошибка
Line number	Какой оператор в пакете или процедуре генерировал ошибку

- В SQL Server (не Azure SQL Database):
  - Сообщения об ошибках в **sys.messages;**
  - Можно добавлять пользовательские сообщения через **sp_addmessage.**

Выбрасывание ошибок. RAISERROR

- Команда RAISERROR
  - Выбрасывание ошибок, определенных в **sys.messages** (только SQL Server)
  - Выбрасывание явно указанного сообщения, кода серьезности и внутреннего состояния (SQL Server и Azure SQL Database)

```
RAISERROR (‘Произошла ошибка’, 16, 0);
```

```
RAISERROR (N’Это сообщение %s %d.', -- Текст сообщения.
           10, -- Код серьезности (Severity),
           1, -- Код состояния (State),
           N’номер', -- Первый аргумент.
           5); -- Второй аргумент.
-- Текст сообщения: Это сообщение номер 5.
```

Выбрасывание ошибок. THROW

- Команда THROW:
  - Замена для RAISERROR;
  - Выбрасывание явного номера ошибки, сообщения и состояния (серьезность = 16);
  - Повторное выбрасывание (пробрасывание) существующей ошибки.

```
THROW 50001, ‘Произошла ошибка‘, 0;
```

```
EXEC sys.sp_addmessage @msgnum = 60000, @severity = 16,
@msgtext = N'Это тестовое сообщение с числовым параметром (%d),
строковым параметром (%s) и другим строковым параметром (%s).',
@lang = 'us_english'  --, @replace = 'replace';
GO

DECLARE @msg NVARCHAR(2048) = FORMATMESSAGE(60000, 500, N'Первая
строка', N'вторая строка');
THROW 60000, @msg, 1;
```

Обработка ошибок

- Используйте оператор TRY…CATCH;
- Обработка ошибок в CATCH:
  - Получить информацию об ошибке:
    - @@ERROR
    - ERROR_NUMBER()
    - ERROR_MESSAGE()
    - ERROR_SEVERITY()
    - ERROR_STATE()
    - ERROR_PROCEDURE()
    - ERROR_LINE()
  - Выполнить коррекцию или журналирование;
  - Пробросить исходную ошибку или выбросить новую пользовательскую ошибку.

```
DECLARE @Discount INT = 0;
BEGIN TRY
        UPDATE SalesLT.Product
        SET Price = Price / @Discount
END TRY
BEGIN CATCH
        PRINT ERROR_MESSAGE();
        THROW 50001, 'Произошла
ошибка', 0;
END CATCH;
```

Введение в транзакции

- Транзакция – группа задач, представляющих единое целое.
- Все задачи транзакции должны вместе успешно выполниться или не выполниться; частичное выполнение не разрешено.

```
-- Две задачи, составляющие неделимую единицу:
INSERT INTO SalesLT.SalesOrderHeader ...
INSERT INTO SalesLT.SalesOrderDetail ...
```

- Отдельные операторы изменения данных автоматически трактуются как автономные транзакции.
- SQL Server использует механизм блокировок и журналирование изменений для поддержки транзакций.

Реализация явных транзакций

- Используйте **BEGIN TRANSACTION** для начала транзакции.
- Используйте **COMMIT TRANSACTION** для завершения транзакции.
- Используйте **ROLLBACK TRANSACTION** для отмены транзакции.
  - Или включите опцию XACT_ABORT для автоматической отмены транзакции при ошибках
- Используйте @@TRANCOUNT и XACT_STATE() для проверки состояния транзакции.

```
BEGIN TRY
  BEGIN TRANSACTION
    INSERT INTO SalesLT.SalesOrderHeader…
    INSERT INTO SalesLT.SalesOrderDetail…
  COMMIT TRANSACTION
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0
  BEGIN
    ROLLBACK TRANSACTION
  END
  PRINT ERROR_MESSAGE();
  THROW 50001, 'Произошла ошибка', 0;
END CATCH;
```

Реализация транзакций @@TRANCOUNT

- количество операторов BEGIN TRANSACTION выполненных в текущей сессии (без парных подтверждений или отмен транзакций).

```
PRINT @@TRANCOUNT
--  Оператор BEGIN TRAN увеличит счетчик транзакций на 1.
BEGIN TRAN
   PRINT @@TRANCOUNT
   BEGIN TRAN
      PRINT @@TRANCOUNT
-- Оператор COMMIT уменьшит счетчик транзакций на 1.
   COMMIT
   PRINT @@TRANCOUNT
COMMIT
PRINT @@TRANCOUNT
```

```
--
Результаты:
--0
--1
--2
--1
--0
```

Особенности работы:
https://stackoverflow.com/questions/12456579/rollback-the-inner-transaction-of-nested-transaction

Тонкости отката “вложенных” транзакций

- В SQL Server есть только одна транзакция в единицу времени и у нее есть счетчик @@TRANCOUNT (begin transaction увеличивает его, а commit transaction - уменьшает). Только commit приводящий счетчик к 0 фиксирует транзакцию.
- rollback transaction – откатывает транзакцию и очищает @@TRANCOUNT.

```
create table #t (col1 int)

insert #t (col1) values (1)
BEGIN TRANSACTION
update #t set col1 = 2 -- Это откатывается
BEGIN TRANSACTION
update #t set col1 = 3 -- Это тоже откатывается
ROLLBACK TRANSACTION
update #t set col1 = 4 -- Это запускается ВНЕ транзакции
COMMIT TRANSACTION -- Выбрасывает ошибку - нет текущей транзакции
-- в #t содержится строка со значением 4
--select * FROM #t-- 4
```

The COMMIT TRANSACTION request has no corresponding BEGIN TRANSACTION

12 Курсоры

Курсор

- именованная область памяти, содержащая результирующий набор SELECT-запроса.



ProductID	ProductName	UnitPrice	RankByPrice
8	Gizmo	263.50	1
29	Widget	123.79	2
9	Thingybob	97.00	3

Использование курсоров. Базовые шаги

- Использование оператора **DECLARE** для создания курсора и определения SQL-оператора возвращающего записи.
- Использование оператора **OPEN** для запроса данных.
- Использование оператора **FETCH** для получения записей из курсора для дальнейшей обработки.
- Использование оператора **CLOSE** для закрытия курсора, когда он более не нужен.
- Использование оператора **DEALLOCATE** для удаления ссылки на курсор.

Переменные для курсоров

- **@@Fetch_Status** (глобальная для всех курсоров в текущей сессии):
- -2 – означает, что строка была удалена;
- -1 – означает положение за последней строкой (нет данных).
- **@@Cursor_Rows**
- Возвращает количество строк, доступное для последнего открытого в данной сессии курсора;
- Возвращает -1, если курсор динамический.

Опции FETCH

- Правила при создании курсоров с использованием расширений Transact-SQL DECLARE:
  - Если указан FORWARD_ONLY или FAST_FORWARD, то NEXT – единственная доступная опция для оператора FETCH.
  - Если опции DYNAMIC, FORWARD_ONLY или FAST_FORWARD не указаны, и указана одна из опций KEYSET, STATIC или SCROLL, то все опции FETCH поддерживаются.
  - Курсоры DYNAMIC SCROLL поддерживают все опции FETCH, кроме ABSOLUTE.

NEXT

PRIOR

FIRST

LAST

ABSOLUTE

RELATIVE

Типы курсоров

- Чувствительность к изменениям базы данных:
  - Dynamic
    - Полная – Все изменения видны, но ценой больших накладных расходов
  - Keyset
    - Частичная – да, но только для обновлений и удалений.
  - Static
    - Нет – работает на копии данных (из TempDB).

Пример работы с курсором

```
declare @Name nvarchar(50), @ListPrice money
declare c1 cursor LOCAL FAST_FORWARD for
       select Name, ListPrice from SalesLT.Product ORDER BY ListPrice

open c1

while (1=1)
begin
       fetch c1 into @Name, @ListPrice
       if @@fetch_status <> 0 break
       PRINT @Name + ' - $' + CAST (@ListPrice as varchar(16))
end

close c1
deallocate c1
```

Хранимые процедуры для работы курсорами

sp_cursor_list	sp_cursorprepare	sp_cursor sp_cursorprepexec
sp_cursorclose	sp_cursorunprepare	sp_cursorexecute
sp_describe_cursor	sp_cursorfetch	sp_describe_cursor_columns
sp_cursoropen	sp_describe_cursor_tables	sp_cursoroption

Системные храним. проц.	Описание
sp_cursor_list	Возвращает список курсоров, доступных для соединения в настоящий момент времени, а также их атрибуты.
sp_describe_cursor	Описывает атрибуты курсора, например имеет ли он тип "forward-only" или "scrolling".
sp_describe_cursor_columns	Описывает атрибуты столбцов результирующего набора.
sp_describe_cursor_tables	Описывает базовые таблицы, к которым курсор получает доступ